Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique

TP ALGO3

Support TP programmation C - l'allocation dynamique -

Fait par :

Nom et prénom : ABDELMALEK BENMEZIANE

Contents

1 L'allocation dynamique en C **1**

1.1 Définition de l'allocation 1

1.2 Les types d'allocation et différence entre eux 1

1.3 L'allocation dynamique 1

1.4 Les fonctions de l'allocation dynamique 2

1.4.1 `malloc()` 2

1.4.2 `calloc()` 2

1.4.3 `realloc()` 2

1.4.4 `free()` 2

1.5 Exemples 2

1.5.1 Exemple avec `malloc()` 2

1.5.2 Exemple avec `calloc()` 3

1.5.3 Exemple avec `realloc()` 4

1.6 Conclusion 6

Chapter 1

L'allocation dynamique en C

1.1 Définition de l'allocation

L'allocation mémoire est le processus par lequel un programme réserve un espace en mémoire vive (RAM) pour stocker des données. En langage C, cela peut se faire de manière statique ou dynamique, selon le moment où la mémoire est réservée.

1.2 Les types d'allocation et différence entre eux

Type	**Moment**	**Durée**	**Taille connue**	**Libération**
Statique	Compilation	Tout le programme	Oui	Non
Dynamique	Exécution	Manuelle	Non	Oui

1.3 L'allocation dynamique

L'allocation dynamique permet de réserver de la mémoire pendant l'exécution du programme, selon les besoins réels. Elle permet une gestion flexible de la mémoire.

Elle est souvent utilisée pour :

- Gérer des structures de données dynamiques (listes, piles, files, arbres,...)

- Allouer de la mémoire selon des entrées utilisateur

1.4 Les fonctions de l'allocation dynamique

Les fonctions sont disponibles dans la bibliothèque `stdlib.h`.

1.4.1 `malloc()`

Alloue un bloc mémoire non initialisé.

```
int *ptr = (int*) malloc(5 * sizeof(int));
```

1.4.2 `calloc()`

Alloue et initialise à zéro un bloc mémoire.

```
int *ptr = (int*) calloc(5, sizeof(int));
```

1.4.3 `realloc()`

Redimensionne un bloc mémoire existant.

```
ptr = (int*) realloc(ptr, 10 * sizeof(int));
```

1.4.4 `free()`

Libère la mémoire allouée dynamiquement.

```
free(ptr);
```

Remarque : Toujours vérifier que les fonctions d'allocation ne retournent pas `NULL`. Après un `free`, il est recommandé de mettre le pointeur à `NULL` pour éviter les erreurs.

1.5 Exemples

1.5.1 Exemple avec `malloc()`

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *tab;
    int n = 5;

    // Allocation avec malloc()
    tab = (int*) malloc(n * sizeof(int));
    if (tab == NULL) {
        printf(" chec  d'allocation\n");
        return 1;
    }

    // Modification de valeur
    for (int i = 0; i < n; i++) {
        tab[i] = i * 10;
        printf("%d ", tab[i]);
    }

    // Lib ration de la m moire
    free(tab);
    return 0;
}
```

1.5.2 Exemple avec `calloc()`

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *tab;
    int n = 5;

    // Allocation avec calloc()
    tab = (int*) calloc(n, sizeof(int));
    if (tab == NULL) {
        printf(" chec  d'allocation\n");
        return 1;
```

```
    }

    // Affichage des valeurs initialis es
    printf("Valeurs apr s calloc (initialis es    0) :\n");
    for (int i = 0; i < n; i++) {
        printf("tab[%d] = %d\n", i, tab[i]);
    }

    // Modification des valeurs
    for (int i = 0; i < n; i++) {
        tab[i] = i + 1;
    }

    // Affichage apr s modification
    printf("Valeurs apr s modification :\n");
    for (int i = 0; i < n; i++) {
        printf("tab[%d] = %d\n", i, tab[i]);
    }

    // Lib ration de la m moire
    free(tab);
    return 0;
}
```

1.5.3 Exemple avec `realloc()`

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *tab;
    int n = 5;

    // Allocation initiale
    tab = (int*) malloc(n * sizeof(int));
    if (tab == NULL) {
        printf(" chec  d'allocation\n");
        return 1;
    }
```

```c

    // Initialisation
    for (int i = 0; i < n; i++) {
        tab[i] = i + 1;
    }

    // Affichage initial
    printf("Tableau initial : ");
    for (int i = 0; i < n; i++) {
        printf("%d ", tab[i]);
    }
    printf("\n");

    // R allocation (agrandir    10    lments   )
    int nouveau_n = 10;
    int *temp = (int*) realloc(tab, nouveau_n * sizeof(int));
    if (temp == NULL) {
        printf(" chec  de r allocation\n");
        free(tab); // Lib rer l'ancienne m moire
        return 1;
    }
    tab = temp;

    // Initialisation des nouvelles cases
    for (int i = n; i < nouveau_n; i++) {
        tab[i] = (i + 1) * 10;
    }

    // Affichage apr s  r allocation
    printf("Tableau apr s realloc : ");
    for (int i = 0; i < nouveau_n; i++) {
        printf("%d ", tab[i]);
    }
    printf("\n");

    // Lib ration de la m moire
    free(tab);
    return 0;
}
```

1.6 Conclusion

L'allocation dynamique en C permet une gestion flexible et efficace de la mémoire à l'exécution. Elle est indispensable pour les structures de données complexes. Toutefois, elle doit être utilisée avec rigueur pour éviter les fuites mémoire ou les erreurs d'accès.